GENOMIC SEQUENCE CLASSIFICATION

Simplified end-to-end architecture | Five main stages for the complete project

1. INPUT DATA	2. PREPROCESSING	3. REPRESENTATION	4. MODELS & PREDICTION	5. EVALUATION & OUTPUT
Sources and user access	Prepare and validate	Convert DNA into model inputs	Train, select and reuse	Measure and present results

A. PANGENOME -> DNABERT WORKFLOW

Graph + labels + checkpoint

GFA / rGFA pangenome graph
CSV / TSV / VCF annotations
Local / Hugging Face checkpoint
Optional ClinVar demo graph helper

Entry point: main.py command line

Graph to DNA sequences

Parse graph; build directed graph
Contract cycles into a DAG
Extract and deduplicate paths
Concatenate path DNA sequences

Graph processing + path extraction

Checkpoint-compatible tokens

Overlapping DNA k-mers
Checkpoint vocabulary token IDs
Special tokens + pad / truncate
Input IDs + attention masks

Tokenization + sequence encoding

DNABERT binary classifier

Grouped train / validation / test
Fine-tune BERT + classification head
Choose threshold on validation
Reuse trained model for inference

Output: BENIGN / PATHOGENIC

Test metrics + saved experiment

Accuracy, precision, recall and F1
ROC-AUC, AP and confusion matrix
Save model, tokenizer, threshold
Record splits, hashes and metrics

Console predictions + experiment.json

A and B are two execution paths within one project.

B. FASTA -> CLASSICAL ML WORKFLOW

FASTA dataset + user upload

APP / PSEN1 / PSEN2 sequences
Healthy / Unhealthy folder labels
Browser settings or experiment CLI
New FASTA upload + selected gene

Flask dashboard or gene_classifier.py

Sequence checks + data split

Load and clean training FASTA
Audit duplicates and label conflicts
Gene / class-stratified holdout
Five-fold CV on training data only

Upload: validate bases, size and length

Fold-fitted feature pipeline

K-mers + nucleotide composition
Mutation / context + gene features
Training-fold healthy references
Scaling; optional selection / SMOTE

Reuse fitted transforms for prediction

Classical ML classifiers

Random Forest / SVM / LR / GB
Select by training CV macro F1
Refit winner; retain web pipeline
Predict uploads without retraining

Output: Healthy / Unhealthy

Held-out results + dashboard

Metrics, per-gene results and curves
Confusion matrix + model comparison
Feature importance when available
Per-record predictions + probabilities

Browser JSON / dashboard; CLI plot files